
📘 **ADR-004**

Stable Service Contract

C3 Contract Control Center

Architecture Decision Record

Status: Approved

Date: 26 June 2026

Decision ID: ADR-004

Purpose

This Architecture Decision Record formalizes the service abstraction used throughout C3.

The objective is to ensure that application components remain completely independent of the underlying data source, allowing the platform to transition between mock data, SharePoint, future APIs, or other storage providers without requiring changes to the user interface or business logic.

This establishes the Service Layer as a permanent architectural boundary within the platform.

Context

During early development, C3 consumed mock datasets directly through a lightweight SharePoint service.

As the project evolved toward production SharePoint integration, it became clear that the application required a stable contract separating:

- User Interface
- Business Logic
- Data Retrieval

Without this separation, every future infrastructure change would require modifications across multiple screens and components.

The platform therefore introduced a dedicated Service Interface and Adapter pattern.

Decision

All application functionality shall retrieve data exclusively through the SPService interface.

No React component, screen, hook, or business rule may access SharePoint, APIs, or external data sources directly.

Instead, every request must follow the standardized architecture:

React Components

│

▼

React Hooks

│

▼

SPService Interface

│

▼

Service Adapter

│

▼

External Data Source

Approved Architecture

The following service flow is adopted as the official architecture:

Contracts Workspace

│

▼

useContracts()

│

▼

```
useSP()
   │
   ▼
createSPService()
   │
   ▼
┌──────────────────────────┐
│ Mock Adapter        │
│ SharePoint Adapter    │
│ Future API Adapter     │
└──────────────────────────┘
```

Benefits

This architecture provides:

Data Source Independence

The user interface remains unchanged regardless of where data originates.

Supported implementations include:

- Mock Data
- SharePoint
- REST APIs
- SQL
- Dataverse
- Future Intelligence Platform Services

Simplified Testing

Mock implementations can be substituted without modifying screens.

This enables:

- Local development

- Unit testing
- UI validation
- Offline development

Infrastructure Flexibility

Future migrations require changes only within the adapter implementation.

Examples include:

- Mock → SharePoint
- SharePoint → API
- API → Dataverse

Application screens remain unaffected.

Improved Maintainability

Responsibilities are clearly separated.

Layer	Responsibility
Components	Presentation
Hooks	Data orchestration
Service Interface	Business contract
Adapter	Infrastructure implementation
Data Source	Storage

Consequences

Positive

- Stable architecture
- Reduced coupling
- Easier testing
- Simplified infrastructure migrations

- Cleaner business logic
- Improved long-term maintainability

Trade-offs

- Additional abstraction layer
- Slight increase in implementation complexity
- Adapter maintenance required for each supported data source

These trade-offs are considered acceptable given the platform's long-term scalability objectives.

Implementation

Current adapters include:

- Mock SharePoint Adapter
- SharePoint Adapter (implementation in progress)

Future adapters may include:

- REST API Adapter
- Dataverse Adapter
- Integration Platform Adapter

No changes to React components are required when introducing additional adapters.

Architecture Principle

User interface components must remain completely unaware of the underlying data source.

The Service Interface represents the single approved entry point for all platform data access.

Decision

Approved

The Stable Service Contract is adopted as a permanent architectural principle of the C3 Contract Control Center and shall apply to all current and future application domains.
